

BEN FOOTE

Principal Software Engineeer

CONTACT

424-236-3266

ben.foote@gmail.com

7270 Martin Wright Rd,

Westfield, NY 14787

SKILLS

- Software Team Leadership
- Agile/Scrum Master
- Technical Documentation
- Full SDLC
- Decision Making
- Customer Communications
- End User Training
- MISRA-C
- DO-178C

SOFTWARE

- Atlassian Stack
- Docker
- Azure
- API Design (REST, gRPC, GraphQL, MQTT)
- LLM's (CoPilot and Claude)
- Linux
- Visual Studio/VSCode/Rider

LANGUAGES

- Assembly (MIPS and x86)
- C#/VB.NET/.NET Stack
- C/C++
- TypeScript
- Java

ABOUT ME

I am a versatile, flexible, and accomplished Principal Software Engineer with 20+ years of experience. I specialize in modernizing legacy systems and implementing Agile methodologies. I have a proven track record in leading teams through significant growth, enhancing existing software platforms with modern patterns such as microservices, and delivering high quality solutions for the commercial, government, and defense sectors. I've acted as a technical lead on making technical decisions that guide software solutions, and hold a US Patent for a novel battery charging implementation. I hold a Top Secret Security Clearance. I also specialize in UI/UX work, as I'm passionate about a good experience. I want to find challenging work that fits kills, and continue to learn new technologies and collaborate as part of a team.

EXPERIENCE

PacStar - A Curtiss-Wright Company - Portland, OR

Principal Software Engineer 2024 – 2026

Helped build a greenfield enterprise software product that would leverage the existing device support in our existing NMS, IQ-Core, as well as added many new capabilities. This included using Docker containers to leverage a microservices architecture, switching from a desktop application to a browser-based Blazor application, and used SignalR and Kestrel to publish a REST API to allow interoperability with third party software vendors.

Senior Software Engineer 2017 - 2024

Helped modernize a legacy C# WPF desktop application by breaking the monolithic code base into a more microservices oriented architecture, all while adding new features and capabilities. The software, IQ-Core, is an Network Management System (NMS) application akin to SolarWindows or Splunk, but delivers much more value for the price to customers.

Saft - Space & Defense Division - Hunt Valley, MD

Senior Software Engineer 2015- 2017

Modernized the software development processes at Saft - shepherded the move from waterfall to Agile/Scrum. Also introduced cross platform development using Xamarin/C# for desktop development as well as mobile, and moved the legacy Battery Management System (BMS) from a custom 16-bit SOC, to an commercial embedded Linux SBC running Linux/FreeRTOS. Also designed and delivered automated test hardware and software to increase product delivery velocity.

Software Engineer 2009- 2015

Worked on a small IPT team to help deliver Saft's first mass produced vehicle battery that replaced existing 6T lead acid batteries on various vehicle platforms. This involved developing, from scratch, embedded firmware that was safety critical with 100% uptime. I also developed our next generation Battery GUI software that worked across multiple product lines and communications protocols.

EDUCATION

Johns Hopkins University 2009 - 2014

Master of Science in Information Systems Engineering

Penn State Erie - The Behrend College 1999 - 2004

Bachelor of Science in Computer Engineering

ADDITIONAL EXPERIENCE

Saft Space & Defense Division - Hunt Valley, MD

Associate Software Engineer 2007 - 2009

Initially worked on the GUI software that communicates to the high power LiON batteries that Saft manufactures. This work involved updating legacy Visual Basic 6 applications to communicate via CAN-Bus for new battery designs and new APIs. After proving successful, I then moved on to working on the 8 and 16 bit embedded safety critical software that helped monitor and control batteries.

Northrop Grumman - Elkridge, MD

Electrical Design Engineer 2004 - 2007

Migrated the French designed Solystic designed Flats Sequencing System to comply with United States electrical and safety standards for the US Postal System, converting power electronics to operate at 60Hz. This involved drafting cable designs, electrical panels, and doing mechanical design. Helped assemble an initial prototype of the American FSS design at Northrop Grumman, which was later disassembled and delivered to a Postal Service testing facility. Traveled to customer facilities, participated in acceptance testing with Postal representatives, and handled the sign-off and submission of FSS purchases.

TECHNICAL ACCOMPLISHMENTS

PacStar - A Curtiss-Wright Company - Portland, OR

API Implementation of the TriLOS (Tri-band Line-of-Sight) Radio

The Army customer, using PacStars IQ-Core software, wished to manage their back haul TriLOS radio using IQ-Core. Normally, when adding new device support to IQ-Core, PacStar works with the device manufacturer to obtain API documents to add device support. With the TriLOS, the manufacturer contract with the Army had ended, and so I was tasked with reverse engineering the protocol in order to support it in IQ-Core. This involved traveling to customer sites, as PacStar was not allowed to posses a TriLOS radio, using Wireshark to capture traffic during various configuration activities, and then implement them in IQ-Core. This process was time consuming, but IQ-Core now contains fully operational support for the TriLOS radio.

Additional information: https://tinyurl.com/trilosradio

PacStar - A Curtiss-Wright Company - Portland, OR

IQ-Core REST API implementation

As per a customer request, a web based REST API was required to be added to IQ-Core. I, and one other engineer, worked with the Army customer to add a REST API to IQ-Core. Previously, IQ-Core had no publicly facing API to allow for 3^{d} party interoperability. We worked along side a competitor to ensure that both NMS systems could interoperate using publicly available REST APIs and were able to successfully demonstrate this functionality to the government. This was an intense 1 week long project that greatly benefited our software and continues to be added to and evolved to this day.

Saft Space & Defense Division - Hunt Valley, MD

Patent for a Novel Battery Charging Implementation

Lithium Ion batteries require careful cell level voltage balancing to remain safe, and provided the maximum capacity to end users. This is normally handled on a per-battery system that can contain many cells. While at Saft, we developed a 12 Volt battery that would automatically balance it's internal cells. However, a need arose to connect 2 of these 12V batteries in series to create a 24 Volt system. The 2 batteries had no way to communicate with each other in a traditional sense via CANBus. Myself, and an electrical engineer came up with a novel way to allow the batteries to vary their voltages between one another to create a rudimentary communication system that would allow the series batteries to charge and remain balanced with one another. As a result, myself and several other engineers were awarded a patent for this design.

Additional information: https://tinyurl.com/batterypatent

Saft Space & Defense Division - Hunt Valley, MD

J1939 CANBus Network Layer Implementation

As part of Saft's Xcelion X6T commercial battery development program, we were required to implement the first 3 layers of the J1939 SAE standard. There were no commercially available implementations for the 8-bit microcontroller we chose, so I was tasked with implementing the Data Link and Network layers of J1939. This was an extremely challenging task, as the only documentation we had was the official SAE handbook, and no reference implementations. Not only did I implement the microcontroller side, I also had to develop a companion Desktop GUI app that spoke the same API to allow for battery communications and control. This implementation was verified with industry standard vehicle communication software.

Saft Space & Defense Division - Hunt Valley, MD

Realtime Pre-emptive OS Scheduler, and J1939 Bootloader

For the Xcelion X6T program, I was tasked with implementing a real-time pre-emptive scheduler for our 8-bit microcontroller. This ensured that individual tasks wouldn't exceed their allotted time slice, and would use minimal CPU overhead while doing so. This also involved the microcontroller going into deep sleep when it was idle, waking up in a medium-power state on an interval to perform cell balancing, and returning to a fully operational state in less than 15mS for high availability when the customer needed to use the battery.

Since the microcontroller required 100% uptime, and had no external programming port, I developed an embedded boot-loader that would allow firmware updates over the J1939 customer communications port. I also developed the accompanying C#/WPF desktop application that would send the updated firmware to the battery. This was a customer facing application, and would allow Saft to send updated firmware to customers whenever it was available.